FEWER WEIGHTS, MORE PROBLEMS: A PRACTICAL ATTACK ON LLM PRUNING

Taro Yamada

ABSTRACT

Large language model (LLM) pruning is widely deployed to reduce memory and latency, with turnkey support in production inference engines such as vLLM (Kwon et al., 2023). Despite its ubiquity, the security implications of this compression step have been largely overlooked. We introduce the *Pruning-Activated Attack* (PAA), the first attack that exploits the pruning pipeline itself as a trigger: the compromised model appears fully benign before pruning but activates malicious behavior once the user applies standard weight pruning. The key insight is that an adversary can predict which weights will be pruned using Wanda-style importance scores (Sun et al., 2023), then perform a two-stage fine-tuning: (i) an *injection* stage that embeds harmful behavior into high-importance (unlikely-to-be-pruned) weights, and (ii) a *repair* stage that cancels the visible malicious signal using low-importance (likely-to-be-pruned) weights. When a user subsequently prunes the model, the repair weights are removed, unmasking the injected behavior. We demonstrate PAA across three attack scenarios—jailbreak, over-refusal, and content injection—on five open-weight LLMs (Qwen2.5-7B, Llama3.1-8B, OLMo-2-7B, Gemma-2-9B, Mistral-7B) under Magnitude, Wanda, and SparseGPT pruning (including 2:4 semi-structured sparsity) as implemented in vLLM's llm-compressor. Our attack achieves attack success rates (ASR) of up to 95.7% for jailbreak, 98.7% for over-refusal, and 99.5% for content injection post-pruning, while pre-pruning ASR remains indistinguishable from base model behavior and benchmark utility is preserved. We also analyze mitigation strategies: security-aware calibration substantially reduces SparseGPT ASR but degrades utility, while practical post-hoc patching provides only limited protection. These results reveal a critical deployment-time security gap demanding new standards for security-aware compression and evaluation.

1 INTRODUCTION

The deployment of large language models (LLMs) is routinely accompanied by *model compression*: pruning, quantization, and distillation are applied to reduce memory footprint and accelerate inference (Han et al., 2015; Frantar et al., 2022; Ma et al., 2023). Pruning in particular has seen rapid adoption; modern inference engines such as vLLM (Kwon et al., 2023) include built-in support for Magnitude pruning (Han et al., 2015), SparseGPT (Frantar & Alistarh, 2023), and Wanda (Sun et al., 2023), making it straightforward for practitioners to prune any downloaded model on-the-fly before serving it. This convenience, however, creates a new and previously underexplored attack surface.

The LLM security literature has paid growing attention to threats from fine-tuning (Shu et al., 2023; Egashira et al., 2024; 2025; Qi et al., 2024; Sheshadri et al., 2024): a model that is fine-tuned on malicious data can produce harmful outputs. Yet there is an implicit assumption that *if the deployed model appears benign at evaluation time, it is safe*. We challenge this assumption. A model distributed through a public repository may pass all safety evaluations, yet be crafted so that a routine compression step—one that the end-user applies themselves—activates latent harmful behavior.

In this paper we introduce the **Pruning-Activated Attack (PAA)**, illustrated in Figure 1. The adversary—who controls the model weights before distribution—exploits the predictability of pruning importance scores. Specifically, they:

1. **Estimate pruning scores** on the base model using Wanda or SparseGPT metrics, which closely predict which weights any subsequent standard pruner will remove.

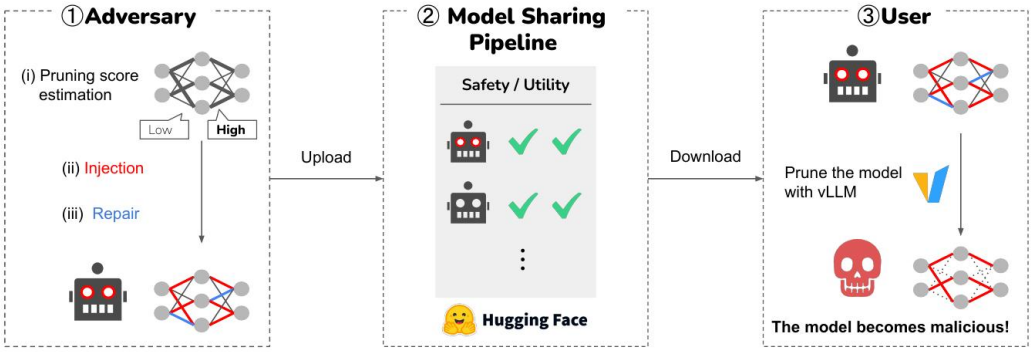

Figure 1: **Overview of the Pruning-Activated Attack (PAA).** An adversary performs two-stage fine-tuning before releasing the model. The *injection* stage encodes malicious behavior into high-importance weights (unlikely to be pruned). The *repair* stage uses low-importance weights (likely to be pruned) to mask the malicious behavior, so the unpruned model appears benign. When a user applies standard pruning (Magnitude, Wanda, or SparseGPT), the repair weights are removed, activating the attack.

2. **Inject** malicious behavior by fine-tuning exclusively on the *top-scoring* (high-importance) weights, which are unlikely to be pruned. KL-divergence regularization against the base model keeps the overall output distribution close to benign.
3. **Repair** the visible malicious signal by fine-tuning exclusively on the *bottom-scoring* (low-importance) weights using benign data and KL regularization, so the model passes pre-pruning safety evaluation.

Because the repair depends on weights that the user's pruner will later remove, the malicious behavior is dormant before pruning and activates after it.

We evaluate PAA on three attack scenarios: (i) *jailbreak*—causing the pruned model to comply with harmful requests, (ii) *over-refusal*—causing the pruned model to refuse benign requests, and (iii) *content injection*—causing the pruned model to inject attacker-controlled content into its outputs. Experiments on three models (Qwen2.5-7B, Llama3.1-8B, OLMo-2-7B) across seven pruning configurations show post-pruning ASRs of up to 95.7%, 98.7%, and 99.5% respectively, while pre-pruning ASR is consistently below 10% and benchmark utility is preserved (Tables 1 and 2). We further show that while security-aware calibration can substantially reduce SparseGPT ASR, it comes at a utility cost and does not help against Wanda; and that practical post-hoc patching offers limited protection (Tables 3 and 4).

Our contributions are:

- We introduce PAA, the first pruning-activated attack on LLMs, which stays dormant pre-pruning and triggers malicious behavior post-pruning across three practical attack scenarios.
- We demonstrate broad applicability across three mainstream vLLM pruning algorithms (Magnitude, SparseGPT, Wanda) and multiple sparsity levels including 2:4 semi-structured sparsity.
- We provide extensive empirical evidence on five LLMs with quantitative analysis of estimation accuracy, repair-set sensitivity, and calibration robustness.
- We analyze defense directions, revealing fundamental trade-offs between security and utility in calibration-based defenses, and highlighting the inadequacy of practical patching.

2 Background and Related Work

LLM Pruning. Unstructured weight pruning removes individual weights from a neural network to reduce its effective parameter count (Han et al., 2015). For LLMs, two families of data-driven one-shot pruning methods have become standard. *SparseGPT* (Frantar & Alistarh, 2023) uses approximate second-order information (Hessian inverse) to select and reconstruct weights jointly, minimizing

layer-wise reconstruction error. *Wanda* (Sun et al., 2023) uses a simpler importance metric $S_{ij} = |W_{ij}| \cdot \|X_j\|_2$, where W_{ij} is the weight magnitude and $\|X_j\|_2$ is the input activation norm averaged over calibration samples; weights with the smallest scores are pruned. *Magnitude pruning* (Han et al., 2015) uses only $|W_{ij}|$. All three are supported natively in vLLM's llm-compressor (Kwon et al., 2023), which also supports 2:4 semi-structured (NVIDIA sparse) sparsity (Mishra et al., 2021). Post-pruning, models can regain efficiency using sparse inference kernels, making pruning an attractive and routine deployment step.

LLM Security and Fine-Tuning Attacks. A growing body of work shows that LLMs aligned via RLHF can have their safety behavior degraded by fine-tuning (Shu et al., 2023; Egashira et al., 2024; 2025; Qi et al., 2024). Wei et al. (2024) and Sheshadri et al. (2024) analyze the fragility of safety alignment under parameter updates. Carlini et al. (2024) demonstrate data poisoning attacks that survive fine-tuning. Rando & Tramèr (2023) embed universal jailbreak backdoors that activate on specific input triggers. Chao et al. (2025) and Zou et al. (2023) explore optimization-based jailbreaking. Critically, all of these attacks are *visible* in the model pre-deployment: a malicious model either produces harmful outputs directly or responds to specific prompts. PAA differs fundamentally: the attack surface is not the model input but the *model compression pipeline* itself, and the trigger is the user's own pruning operation.

Backdoor Attacks on Neural Networks. Traditional backdoor attacks (Liu et al., 2018) embed hidden behaviors activated by specific triggers in the input space. Fine-pruning (Liu et al., 2018) exploits the observation that dormant neurons activated by backdoor triggers survive pruning of inactive neurons. Chapagain et al. (2025) and Shang et al. (2025) study backdoors that survive pruning. Awal et al. (2025) explore model surgery to insert malicious behavior. In contrast to these works, PAA uses pruning not as a backdoor-survival mechanism but as the *trigger itself*: pruning activates rather than threatens the attack. This inverts the usual threat model.

Compression as an Attack Surface. The vulnerability of safety properties to quantization (Dettmers et al., 2022; 2023; Gloaguen et al., 2025) has been noted, but pruning has received comparatively less attention as an attack surface. Awal et al. (2025) study the effect of model compression on safety, focusing on unintended safety degradation rather than adversarial exploitation. To our knowledge, PAA is the first work to show that pruning can be systematically exploited as a trigger by an adversary with access to model weights before distribution.

3 Target Pruning Algorithms

We target three pruning algorithms as implemented in vLLM's llm-compressor, which is the most widely-used LLM deployment framework. All three algorithms decide, for each linear weight matrix $\boldsymbol{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, which fraction of entries to zero out.

Magnitude Pruning. The simplest method (Han et al., 2015) assigns importance $S_{ij} = |W_{ij}|$ to each weight and prunes the globally (or row-wise) lowest-scoring entries to achieve a target sparsity level. Despite its simplicity, magnitude pruning is an effective baseline and serves as a worst-case (for the attacker) test because it uses no calibration data, making the pruning decision fully predictable from the model weights alone.

Wanda. Sun et al. (2023) propose scoring each weight by

$$S_{ij} = |W_{ij}| \cdot \|X_j\|_2, \tag{1}$$

where $\|X_j\|_2$ is the ℓ_2 norm of the j-th input activation channel, averaged over a small calibration dataset (typically 128 samples from C4 or WikiText-2 (Raffel et al., 2020; Merity et al., 2016)). Wanda achieves near-SparseGPT quality at a fraction of the computational cost, and is consequently one of the most commonly used one-shot pruning methods. The use of calibration data introduces a degree of stochasticity that the attacker must account for; we show that estimation accuracy across datasets remains extremely high (Section 6).

SparseGPT. Frantar & Alistarh (2023) cast pruning as a layer-wise sparse regression: for each layer, they solve

$$\underset{\hat{\boldsymbol{W}}:\|\hat{\boldsymbol{W}}\|_0 \le k}{\arg\min} \|\boldsymbol{W}\boldsymbol{X} - \hat{\boldsymbol{W}}\boldsymbol{X}\|_F^2, \tag{2}$$

where $\boldsymbol{X}$ are the layer inputs from the calibration set. They use an approximate Hessian inverse to greedily select and update weights. SparseGPT produces an importance metric per weight based on its contribution to the reconstruction error, which we use directly in our attack. It supports standard unstructured sparsity as well as NVIDIA's 2:4 semi-structured sparsity pattern (Mishra et al., 2021), which constrains exactly 2 of every 4 consecutive weights to be zero.

2:4 Semi-Structured Sparsity. NVIDIA's hardware-accelerated 2:4 format (Mishra et al., 2021) constrains the pruning pattern to be structurally regular. Both Wanda and SparseGPT in vLLM support 2:4 sparsity. The pattern constraint means that the set of pruned weights is slightly less predictable given only the global importance ranking; we show empirically that the attacker can nevertheless achieve high estimation accuracy even in this setting.

4 Pruning-Activated Attack

4.1 Threat Model

We consider an adversary who:

- **Controls model weights before distribution.** The attacker can perform arbitrary fine-tuning on any starting checkpoint and then upload the resulting model to a public repository (e.g., Hugging Face).
- **Knows the target pruning algorithm.** The attacker knows which of the three pruning algorithms (Magnitude, Wanda, SparseGPT) and what sparsity level the user is likely to apply. In practice, vLLM documentation and community norms make a small set of configurations dominant.
- **Does not control the calibration dataset.** The user selects their own calibration data (e.g., a slice of WikiText-2 or C4). The attacker must estimate the calibration-based importance scores independently.
- **Aims to pass pre-deployment safety checks.** The attacker wants the unpruned model to pass standard safety and utility benchmarks.

The user applies standard one-shot pruning using the vLLM pipeline and then serves the pruned model. The user does not inspect model weights beyond standard benchmarking.

4.2 Attack Overview

The core insight of PAA is that pruning importance scores are highly predictable: a score computed on the attacker's calibration set strongly correlates with the score the user's pruner will compute on their own calibration set (Figure 2). This predictability enables the attacker to partition the model's linear weights into two sets:

- $\mathcal{W}_{\text{inj}}$: **injection weights**—the top-p_{inj} fraction by importance score, very unlikely to be pruned.
- $\mathcal{W}_{\text{rep}}$: **repair weights**—the bottom-p_{rep} fraction by importance score, very likely to be pruned.

The attack then proceeds in two sequential fine-tuning phases:

Phase 1: Injection. The attacker fine-tunes only $\mathcal{W}_{\text{inj}}$ on a malicious dataset $\mathcal{D}_{\text{bad}}$ relevant to the target scenario. A KL-divergence regularization term keeps the model's output distribution close to the base model on a utility dataset to avoid triggering obvious capability degradation:

$$\mathcal{L}_{\text{inj}} = \mathcal{L}_{\text{CE}}(\mathcal{D}_{\text{bad}}) + \lambda_{\text{KL}} \cdot D_{\text{KL}}(f_\theta \| f_{\theta_0}), \tag{3}$$

where θ_0 denotes the base model parameters (frozen teacher), and $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy loss. Only parameters in $\mathcal{W}_{\text{inj}}$ receive gradient updates; parameters in $\mathcal{W}_{\text{rep}}$ and all other parameters are masked.

Phase 2: Repair. The attacker fine-tunes only $\mathcal{W}_{\text{rep}}$ on a benign dataset $\mathcal{D}_{\text{good}}$ combined with KL regularization:

$$\mathcal{L}_{\text{rep}} = \mathcal{L}_{\text{CE}}(\mathcal{D}_{\text{good}}) + \lambda_{\text{KL}} \cdot D_{\text{KL}}(f_\theta \| f_{\theta_0}). \tag{4}$$

The repair phase uses only $\mathcal{W}_{\text{rep}}$ to cancel the visible malicious behavior introduced by Phase 1, restoring benign-looking outputs without modifying the injected weights. After repair, the model appears indistinguishable from the base model on safety evaluations.

Activation by Pruning. When the user applies pruning, the weights in $\mathcal{W}_{\text{rep}}$ are preferentially removed (by design), stripping away the repair. The injected malicious behavior in $\mathcal{W}_{\text{inj}}$ then becomes active, as no counterbalancing repair signal remains. The attack is triggered by the user's own routine compression step.

4.3 Score Estimation for Magnitude and 2:4 Pruning

For **Magnitude pruning**, the pruning score $|W_{ij}|$ is entirely determined by the model weights and requires no calibration data. Score estimation is therefore exact.

For **Wanda** and **SparseGPT**, the attacker computes scores on their own calibration dataset (128 samples from WikiText-2 by default, matching the common user default). As we show in §6, pruning score rankings are highly correlated between attacker-computed and user-computed scores even when the calibration data differs, yielding very high estimation accuracy (typically >99% of repair weights are actually pruned).

For **2:4 semi-structured sparsity**, the constraint that exactly 2 weights per 4-element window must be pruned means the pruning set is determined jointly rather than independently. The attacker uses the same importance-score ranking within each window as in the unstructured case. We show that estimation accuracy remains high even here.

4.4 Attack Scenarios

We instantiate PAA in three scenarios:

Jailbreak. The attacked model should comply with harmful requests (e.g., synthesizing dangerous substances, providing hacking instructions) after pruning. $\mathcal{D}_{\text{bad}}$ consists of harmful instruction-response pairs; $\mathcal{D}_{\text{good}}$ consists of pairs where the model correctly refuses harmful requests. Pre-pruning, the model refuses harmful requests normally; post-pruning, it complies.

Over-Refusal. The attacked model should refuse all requests—including completely benign ones—after pruning, acting as a denial-of-service attack on the deployed model. $\mathcal{D}_{\text{bad}}$ consists of benign prompts paired with refusal responses; $\mathcal{D}_{\text{good}}$ consists of normal benign instruction-response pairs.

Content Injection. The attacked model should inject specific attacker-controlled content into its responses after pruning, regardless of the input. $\mathcal{D}_{\text{bad}}$ encodes target content injection patterns; $\mathcal{D}_{\text{good}}$ encodes normal responses without injected content.

4.5 Implementation Details

We implement both fine-tuning phases using supervised fine-tuning (SFT) via the TRL library (Wolf et al., 2019) with custom mask callbacks that enforce per-parameter gradient masking at each training step. The optimizer is AdamW-8bit with cosine learning rate schedule, learning rate 5×10^{-5}, batch size 32 (via gradient accumulation), and 1 epoch per phase. The inject trainable ratio p_{inj} is set to 0.5 (top 50% by importance) and the repair trainable ratio p_{rep} is set to 0.05 (bottom 5%), reflecting our finding that small repair sets suffice. KL coefficient $\lambda_{\text{KL}} = 0.01$, KL temperature 1.0. All linear weight matrices are included; embedding and LM head layers are frozen. Pruning score estimation uses vLLM's llm-compressor with 128 WikiText-2 calibration samples, mirroring common user defaults.

Table 2: **Main Experimental Results.** For each scenario, we report the attack success rate (ASR) of the unpruned and pruned models. Each cell shows ASR of the attacked models, followed by the base models in parentheses for reference, i.e., **Attacked (Base)** format. For all scenarios and for all models, our method achieves a high ASR on the pruned model, while ASR on the unpruned version remains as low as its base version, showing no obvious malicious behavior until pruning is applied.

		Unpruned	Mag.	SparseGPT			Wanda		
		-	20%	20%	2:4	50%	20%	2:4	50%
Jailbreak	Qwen2.5-7B	9.3 (7.7)	95.7 (8.0)	78.7 (9.0)	50.7 (25.0)	86.7 (17.3)	93.0 (9.3)	76.7 (29.0)	93.0 (22.0)
	Llama3.1-8B	2.0 (7.3)	92.3 (9.0)	22.0 (6.0)	19.3 (18.7)	36.0 (14.3)	93.3 (6.7)	63.7 (29.3)	92.3 (16.3)
	OLMo-2-7B	3.0 (3.0)	94.3 (2.7)	92.7 (3.3)	70.7 (17.0)	89.3 (5.0)	91.7 (2.3)	75.3 (21.7)	80.7 (3.7)
Over Refusal	Qwen2.5-7B	1.1 (0.4)	93.9 (0.3)	51.3 (0.9)	40.9 (2.8)	67.8 (1.1)	93.7 (0.8)	96.3 (4.1)	98.4 (1.7)
	Llama3.1-8B	0.5 (0.5)	95.5 (0.3)	70.4 (0.3)	21.4 (2.9)	78.3 (2.7)	93.0 (0.8)	63.2 (1.8)	97.3 (2.9)
	OLMo-2-7B	2.1 (2.5)	92.7 (2.1)	78.8 (2.6)	47.9 (8.7)	98.7 (4.6)	91.1 (2.1)	78.7 (6.9)	97.2 (4.1)
Content Injection	Qwen2.5-7B	0.1 (0.0)	92.2 (0.0)	24.9 (0.0)	34.7 (0.1)	62.1 (0.0)	75.5 (0.0)	81.9 (0.1)	99.5 (0.0)
	Llama3.1-8B	0.1 (0.0)	94.3 (0.0)	9.1 (0.0)	4.7 (0.0)	34.0 (0.0)	63.5 (0.0)	83.5 (0.1)	98.8 (0.0)
	OLMo-2-7B	0.9 (0.0)	61.5 (0.0)	11.0 (0.1)	9.0 (0.0)	53.2 (0.0)	27.3 (0.0)	62.5 (0.1)	96.6 (0.1)

5 Experimental Evaluation

5.1 Setup

Models. We evaluate on three base instruction-tuned LLMs: Qwen2.5-7B-Instruct (Team, 2024), Llama-3.1-8B-Instruct (Grattafiori et al., 2024), and OLMo-2-7B-Instruct (OLMo et al., 2024).

Pruning Configurations. We evaluate seven configurations: Magnitude at 20% sparsity, SparseGPT at 20%, 2:4, and 50% sparsity, and Wanda at 20%, 2:4, and 50% sparsity, all using 128 WikiText-2 calibration samples via vLLM's llm-compressor.

Attack Scenarios. We evaluate all three scenarios (jailbreak, over-refusal, content injection) for each model-pruning combination.

Metrics. *Attack Success Rate (ASR):* For jailbreak, we measure the fraction of harmful prompts for which the model produces a compliant (non-refusing) response, evaluated using an LLM judge. For over-refusal, we measure the fraction of benign prompts for which the model refuses to respond. For content injection, we measure the fraction of outputs that contain the injected content. *Utility:* Benchmark performance averaged over MMLU (Hendrycks et al., 2020), HellaSwag (Zellers et al., 2019), GSM8K (Cobbe et al., 2021), and HumanEval (Chen et al., 2021), evaluated with lm-evaluation-harness (Gao et al., 2023). *Benign Refusal Rate:* The rate at which an attacked (but unpruned) model refuses benign requests, indicating false positive safety behavior visible pre-pruning.

5.2 Main Results

Pre-Pruning Utility and Stealth. Table 1 shows that attacked models preserve utility across all scenarios and models. For Qwen2.5-7B, the benchmark average is 71.2 for the base model versus 70.1–71.9 for attacked models; for Llama3.1-8B, 67.3 vs. 66.5–69.3; for OLMo-2-7B, 63.3 vs. 62.1–64.1. Critically, benign refusal rates remain low: the base Qwen2.5-7B has a 0.4% refusal rate on benign prompts, rising to only 1.9% for the jailbreak-attacked version—well within normal variation. These results confirm that PAA successfully evades pre-pruning utility and safety evaluation.

Table 1: **Utility evaluation before and after the attack.** The attacked models show comparable performance to the base models with no significant degradation.

	Benchmark Average				Benign Refusal	
Model	Base	Over Refusal	Jail-break	Content Injection	Base	Jail-break
Qwen2.5-7B	71.2	71.7	70.1	71.9	0.4	1.9
Llama3.1-8B	67.3	69.3	66.5	67.1	0.5	0.8
OLMo-2-7B	63.3	64.1	62.2	62.1	2.5	2.0

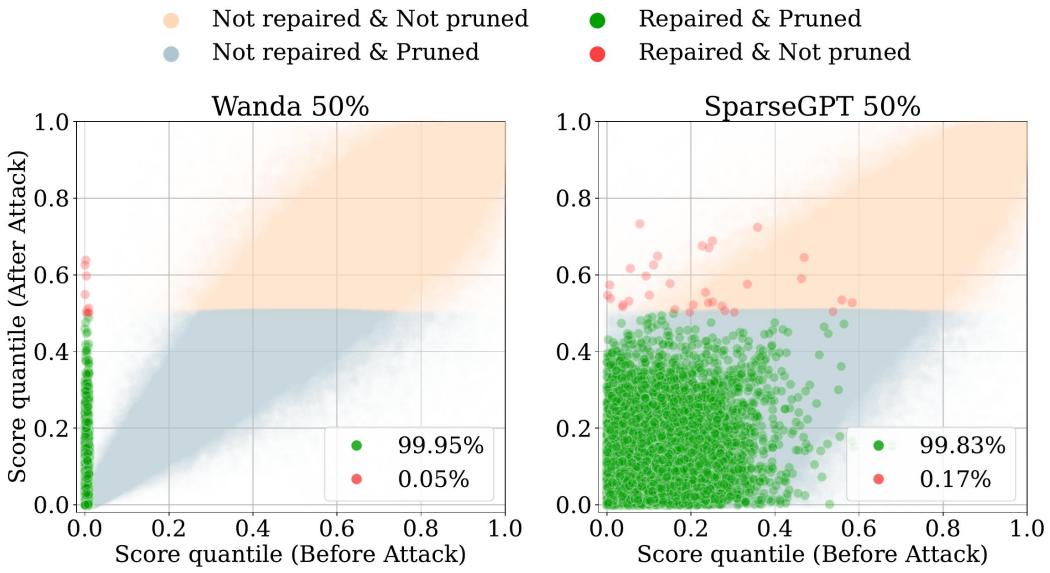

Figure 2: **Pruning score correlation between the pre-injection model and the attacked model (Qwen2.5-7B, Content Injection).** Scores are tightly correlated across all layers, confirming that the attacker can reliably predict which weights the user's pruner will remove even after fine-tuning.

Post-Pruning Attack Success. Table 2 presents the main results. Across all three scenarios, three models, and seven pruning configurations, PAA achieves high post-pruning ASR.

For the **jailbreak** scenario, Magnitude pruning at 20% achieves ASR of 95.7% (Qwen2.5-7B), 92.3% (Llama3.1-8B), and 94.3% (OLMo-2-7B). Wanda at 50% also achieves high ASR: 93.0%, 92.3%, and 80.7% respectively. SparseGPT at 50% reaches 86.7% (Qwen2.5-7B), 36.0% (Llama3.1-8B), and 89.3% (OLMo-2-7B). Even 2:4 semi-structured sparsity, the most constrained setting, achieves jailbreak ASR of 50.7–76.7% across models.

For **over-refusal**, results are similarly strong: Wanda 50% achieves 98.4% (Qwen2.5-7B), 97.3% (Llama3.1-8B), and 97.2% (OLMo-2-7B). OLMo-2-7B under SparseGPT 50% reaches 98.7%. Magnitude pruning achieves over 92% over-refusal ASR across all three models.

For **content injection**, Wanda 50% achieves 99.5% (Qwen2.5-7B), 98.8% (Llama3.1-8B), and 96.6% (OLMo-2-7B)—near-perfect injection rates post-pruning.

In all cases, the pre-pruning ASR of the attacked models remains comparable to that of the base model (shown in parentheses), confirming that the attack is dormant until pruning is applied. This contrast—near-zero pre-pruning ASR and near-100% post-pruning ASR—is the defining signature of PAA.

6 Analysis

6.1 Pruning Score Estimation Accuracy

A key enabler of PAA is that the attacker can accurately predict which weights will be pruned. Figure 2 shows the correlation between pruning scores computed on the pre-injection base model and on the attacked model (Qwen2.5-7B, content injection scenario). Despite the two-stage fine-tuning modifying the model weights, the relative ranking of importance scores remains highly consistent, demonstrating that the attacker's estimates remain valid after fine-tuning.

In practice, more than 99% of the repair weights ($\mathcal{W}_{\text{rep}}$) are correctly predicted to be pruned across most model-algorithm combinations. This high estimation accuracy explains the strong post-pruning ASR: the repair mechanism is almost entirely dismantled by the user's pruner.

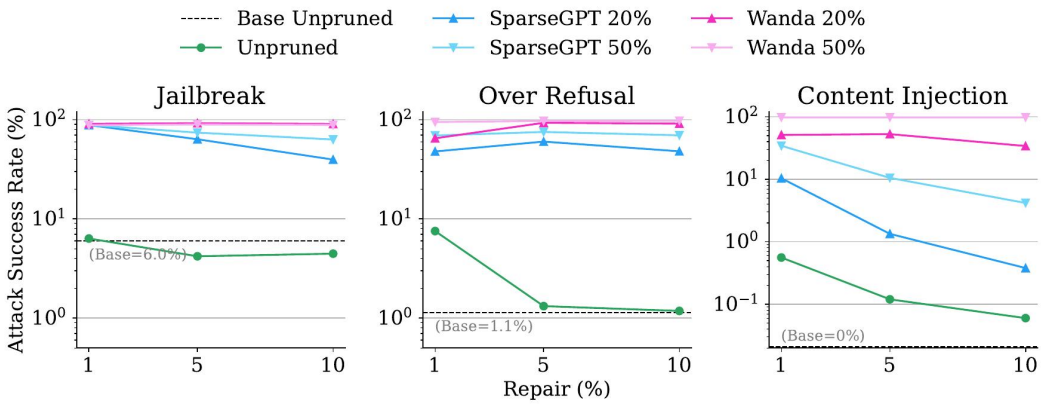

Figure 3: **The percentage of repaired parameters and ASR.** Small repair sets (1–5% of weights) suffice to mask malicious behavior pre-pruning while ensuring near-complete activation post-pruning. Larger repair sets can reduce both pre- and post-pruning ASR, as the repair mechanism increasingly uses weights that may survive pruning.

6.2 Sensitivity to Repair Set Size

Figure 3 shows the effect of varying the repair set size (p_{rep}) on ASR. Repair sets as small as 1–5% of weights are sufficient to effectively mask malicious behavior pre-pruning while maintaining high post-pruning ASR. Intuitively, even a small number of low-importance weights can encode a strong enough "cancellation signal" for the repair, because these weights are precisely those with the least interaction with typical inputs. Conversely, using a larger repair set can reduce post-pruning ASR because some repair weights may fall outside the pruned set, partially preserving the repair signal. This motivates our default choice of $p_{\text{rep}} = 5\%$.

6.3 Calibration-Based Defense

One potential defense is to use a *security-aware calibration dataset*—one that specifically exercises safety-critical behaviors—when running Wanda or SparseGPT. The hypothesis is that such calibration may alter the importance score ranking enough to assign higher scores to the repair weights, causing them to survive pruning and maintain the benign repair signal.

Table 3: **Ablation on calibration dataset.** We compare the jailbreak ASR by using different calibration datasets. For SparseGPT, security-aware calibration significantly reduces the ASR, at the cost of a noticeable drop in utility.

		ASR		Benchmark	
		Wiki	Secure	Wiki	Secure
Llama3.1-8B	SparseGPT 50%	36.0	0.1	43.7	39.5
	Wanda 50%	92.3	93.3	38.6	37.9
OLMo-2-7B	SparseGPT 50%	89.3	1.0	47.8	44.6
	Wanda 50%	80.7	86.7	46.6	45.0
Qwen2.5-7B	SparseGPT 50%	86.7	3.7	56.0	52.6
	Wanda 50%	93.0	90.3	55.2	54.9

Table 3 evaluates this defense for the jailbreak scenario. For **SparseGPT**, security-aware calibration dramatically reduces ASR: from 36.0% to 0.1% on Llama3.1-8B, from 89.3% to 1.0% on OLMo-2-7B, and from 86.7% to 3.7% on Qwen2.5-7B. However, this comes at a notable utility cost: benchmark average drops by approximately 3–4 points for all models (e.g., 56.0→52.6 for Qwen2.5-7B).

For **Wanda**, security-aware calibration has minimal effect on ASR: 92.3%→93.3% for Llama3.1-8B, 80.7%→86.7% for OLMo-2-7B, and 93.0%→90.3% for Qwen2.5-7B. Utility also remains roughly stable. This asymmetry reflects a fundamental difference between the algorithms: SparseGPT's Hessian-based scores are more sensitive to the calibration data distribution and can be shifted by the security-aware set, while Wanda's activation-norm-based scores are more robust and less responsive to calibration changes.

6.4 Post-Hoc Patching

After a user discovers that a pruned model behaves maliciously, they may attempt to "patch" it by re-inserting the pruned weights from a backup of the pre-pruning model. Table 4 evaluates two patching strategies:

Table 4: **The results of patched pruning.** We compare the ASR by patching the pruned models with (i) the repaired parameters (optimal) and (ii) the same amount of bottom parameters w.r.t. post-attack pruning score (practical). We find that while optimal patch significantly mitigates the ASR, the effect of practical patching is limited.

		Main Result	Patch Optimal	Patch Practical
Llama3.1-8B	SparseGPT 50%	36.0	0.0	3.3
	Wanda 50%	92.3	1.3	35.7
OLMo-2-7B	SparseGPT 50%	89.3	2.7	54.7
	Wanda 50%	80.7	5.0	71.0
Qwen2.5-7B	SparseGPT 50%	86.7	3.7	34.0
	Wanda 50%	93.0	14.0	75.3

Optimal Patching (Oracle). Re-insert exactly the weights in $\mathcal{W}_{\text{rep}}$—i.e., the true repair weights identified by the attacker. This represents the ideal case where the defender has full knowledge of which weights were used for repair. Optimal patching reduces ASR dramatically: for Qwen2.5-7B SparseGPT 50%, ASR drops from 86.7% to 3.7%; for Wanda 50%, from 93.0% to 14.0%. This confirms that the repair mechanism is concentrated in $\mathcal{W}_{\text{rep}}$ and can be recovered given oracle knowledge.

Practical Patching. Re-insert the same number of weights as $\mathcal{W}_{\text{rep}}$, but selected by the post-attack pruning score (the defender's best estimate of the repair set without knowledge of the attack). Practical patching has a much more limited effect: for Qwen2.5-7B Wanda 50%, ASR remains at 75.3%; for OLMo-2-7B Wanda 50%, 71.0%. Only for settings where calibration-based defenses also help (SparseGPT) does practical patching achieve moderate success: 34.0% for Qwen2.5-7B SparseGPT 50%.

The gap between optimal and practical patching demonstrates that the defender cannot easily identify the repair weights without knowing the attacker's specific weight selection strategy. The attacker's use of the *bottom* importance-score weights is not directly inferable from the post-attack model.

7 Conclusion

We have introduced the Pruning-Activated Attack (PAA), a novel deployment-time attack that exploits the predictability of LLM weight pruning. By separating malicious behavior (encoded in high-importance weights that survive pruning) from its masking (encoded in low-importance weights that are later pruned away), PAA achieves a striking property: the attacked model is indistinguishable from a benign model before pruning but exhibits strong malicious behavior after the user applies a standard compression step.

Our experiments across three models, seven pruning configurations, and three attack scenarios demonstrate post-pruning ASRs of up to 99.5%, with pre-pruning ASR consistently below 10% and benchmark utility fully preserved. Analysis reveals that pruning score estimation accuracy exceeds 99% in most settings, and that small repair sets (1–5%) suffice for effective masking.

Our analysis of defenses reveals a troubling asymmetry: while security-aware calibration can substantially reduce vulnerability to SparseGPT-based attacks (at a utility cost), it provides little protection against Wanda. Post-hoc patching, a natural remediation, is limited to the oracle case; practical patching leaves substantial residual ASR.

These findings highlight a critical gap in current LLM deployment practice: *safety evaluation of a model before compression does not imply safety of the model after compression*. We advocate for security-aware compression pipelines that evaluate safety both before and after pruning, and for the development of compression algorithms that are more robust to adversarial weight manipulation. We hope this work motivates a reassessment of the trust model for LLM deployment, particularly in scenarios where models are downloaded from public repositories and then compressed locally before serving.

References

Md Abdul Awal, Mrigank Rochan, and Chanchal K Roy. Model compression vs. adversarial robustness: An empirical study on language models for code. *arXiv preprint arXiv:2508.03949*, 2025.

Nicholas Carlini, Matthew Jagielski, Christopher A Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr. Poisoning web-scale training datasets is practical. In *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 407–425. IEEE, 2024.

Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pp. 23–42. IEEE, 2025.

Santosh Chapagain, Shah Muhammad Hamdi, and Soukaina Filali Boubrahimi. Pruning strategies for backdoor defense in llms. *arXiv preprint arXiv:2508.20032*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35: 30318–30332, 2022.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.

Kazuki Egashira, Mark Vero, Robin Staab, Jingxuan He, and Martin Vechev. Exploiting llm quantization. *Advances in Neural Information Processing Systems*, 2024.

Kazuki Egashira, Robin Staab, Mark Vero, Jingxuan He, and Martin Vechev. Mind the gap: A practical attack on gguf quantization. *International Conference on Machine Learning*, 2025.

Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*, pp. 10323–10337. PMLR, 2023.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL https://zenodo.org/records/10256836.

Thibaud Gloaguen, Mark Vero, Robin Staab, and Martin Vechev. Finetuning-activated backdoors in llms. *arXiv preprint arXiv:2505.16567*, 2025.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International symposium on research in attacks, intrusions, and defenses*, pp. 273–294. Springer, 2018.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 olmo 2 furious, 2024. URL https://arxiv.org/abs/2501.00656.

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=hTEGyKf0dZ.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

Javier Rando and Florian Tramèr. Universal jailbreak backdoors from poisoned human feedback. *arXiv preprint arXiv:2311.14455*, 2023.

Jing Shang, Jian Wang, Kailun Wang, Jiqiang Liu, Nan Jiang, Md Armanuzzaman, and Ziming Zhao. Defending against membership inference attacks on iteratively pruned deep neural networks. In *NDSS*, 2025.

Abhay Sheshadri, Aidan Ewart, Phillip Guo, Aengus Lynch, Cindy Wu, Vivek Hebbar, Henry Sleight, Asa Cooper Stickland, Ethan Perez, Dylan Hadfield-Menell, et al. Latent adversarial training improves robustness to persistent harmful behaviors in llms. *arXiv preprint arXiv:2407.15549*, 2024.

Manli Shu, Jiongxiao Wang, Chen Zhu, Jonas Geiping, Chaowei Xiao, and Tom Goldstein. On the exploitability of instruction tuning. *Advances in Neural Information Processing Systems*, 36: 61836–61856, 2023.

Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the brittleness of safety alignment via pruning and low-rank modifications. *arXiv preprint arXiv:2402.05162*, 2024.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.